

Solving Linear and Nonlinear Systems of Equations

Solving Linear Systems

- Applications
- Dense Matrix solution
 - Gaussian-Elimination
 - LU decomposition
 - Crout's method
 - Gauss-Jordan method
- Sparse Matrix Solution
 - Jacobi
 - Gauss-Seidel
 - SOR
 - Krylov methods
 - Sparse Gaussian-Elimination

Applications

- Dense linear systems arising in optimization, BEM, and Spectral methods
- Sparse linear systems resulting from FDA and FEM of ODEs and PDEs

Dense Matrix Solution

- A large number of applications require dense matrix solution
 - E.g. Boundary element method, Method of Moments, Spectral methods.
- Goal: Solve $Ax=b$ for x
- Gaussian Elimination
 - Reduce $Ax=b$ to $Ux=y$ and solve for x using backsubstitution.
- LU factorization
 - Reduce A to LU and solve for x using forward elimination and backsubstitution.
- GE and LU are essentially the same except in GE, the LU factorization and forward elimination steps are combined.
- In LU factorization, the factor LU is stored so that if multiple solutions are needed with the same matrix but different right hand sides it can be accomplished by cheaper forward elimination and backsubstitution operations.

Gaussian Elimination

- Number of operations
 - $(2n^3/3) - (n^2/2)$ subtractions and multiplications
 - $(n^2/2)$ divisions
 - For large n we can ignore n^2 terms so that

$$W = \frac{2n^3}{3}$$

Gaussian Elimination Overview

- Subtract multiples of each row from later rows to make **A** upper triangular
- Solve resulting upper triangular system **U** $x = y$ by backsubstitution

// for each column k

// zero it out below the diagonal by adding multiples of row k to later rows

for k = 1 to n

 // for each row j below row i

 for i = k+1 to n

 // subtract a multiple of row i from row j

 for j = k+1 to n

$A(i,j) = A(i,j) - (A(k,j)/A(k,k)) * A(i,k)$

 endfor

 endfor

 endfor

GE Overview (contd.)

$$a_{0,0}x_0 + a_{0,1}x_1 + \cdots + a_{0,n-1}x_{n-1} = b_0,$$

$$a_{1,0}x_0 + a_{1,1}x_1 + \cdots + a_{1,n-1}x_{n-1} = b_1,$$

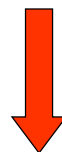
$$\vdots$$

$$\vdots$$

$$\vdots$$

$$\vdots$$

$$a_{n-1,0}x_0 + a_{n-1,1}x_1 + \cdots + a_{n-1,n-1}x_{n-1} = b_{n-1}.$$



$$x_0 + u_{0,1}x_1 + u_{0,2}x_2 + \cdots + u_{0,n-1}x_{n-1} = y_0,$$

$$x_1 + u_{1,2}x_2 + \cdots + u_{1,n-1}x_{n-1} = y_1,$$

$$\vdots$$

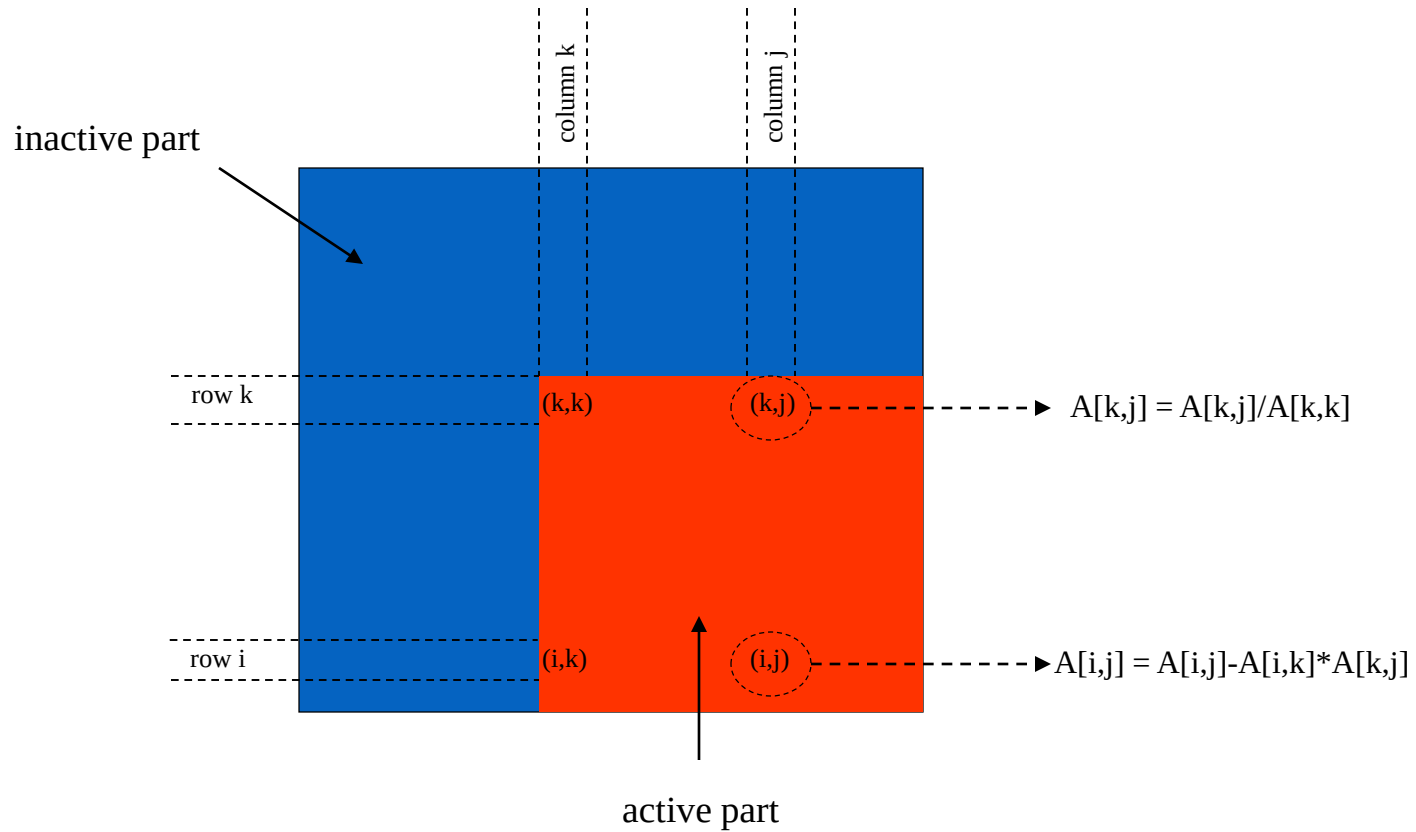
$$\vdots$$

$$x_{n-1} = y_{n-1}.$$

Gaussian Elimination Algorithm

```
// GE Algorithm – make  $Ax=b$  to  $Ux=y$ 
// Computes U and stores into A, computes y
for k=1:n // for each row k
    for j = k+1:n // for all elements right of column k in row k
         $A[k,j] = A[k,j]/A[k,k]$  //divide by diagonal element
    endfor
     $y[k] = b[k]/A[k,k]$  // compute y
     $A[k,k] = 1$ 
    for i = k+1:n // for each row i below row k
        for j = k+1: n // for elements right of column k in row i
             $A[i,j] = A[i,j] - A[i,k]*A[k,j]$  // subtract from row i an  $A[i,k]$  multiple of row k
        endfor
         $b[i] = b[i] - A[i,k]*y[k]$  // update b to compute next y
         $A[i,k] = 0$  //set lower triangular entries of A to zero
    endfor
endfor
for k =n:1
     $x[k]=y[k]$  // back substitute to solve for x
    for i=k-1:1
         $y[i]=y[i]-x[k]*A[i,k]$ 
    endfor
endfor
//end GE
```

GE (contd.)



LU factorization methods

- Goal is to decompose A into two matrices L and U
 - $[A] = [L][U]$
- Once decomposed, solution can be obtained by forward elimination and back substitution
- Two methods are commonly employed
 - Standard method based on Gaussian Elimination
 - Crout's method
- See Gilat and Subramaniam for details

Gauss-Jordan method

- Gauss Jordan method is an elimination method similar to Gaussian elimination
- The original matrix is first augmented with the rhs
 - The matrix portion is then transformed into an identity matrix by:
 - normalizing with the diagonal entries
 - eliminating all entries in the same column below and above each diagonal entry
 - The solution is contained in the modified augmented column
- Detailed steps through an example is given in C & C p. 269-270 & [Gilat and Subramaniam](#)

Gauss-Jordan (Contd.)

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \vdots & b_1 \\ a_{21} & a_{22} & a_{23} & \vdots & b_2 \\ a_{31} & a_{32} & a_{33} & \vdots & b_3 \end{bmatrix}$$

↓

$$\begin{bmatrix} 1 & 0 & 0 & \vdots & b_1^{(n)} \\ 0 & 1 & 0 & \vdots & b_2^{(n)} \\ 0 & 0 & 1 & \vdots & b_3^{(n)} \end{bmatrix}$$

↓

$$\begin{array}{rcl} x_1 & & = b_1^{(n)} \\ & x_2 & = b_2^{(n)} \\ & & x_3 = b_3^{(n)} \end{array}$$

Solving Sparse Linear Systems

- Simple iterative methods
 - Jacobi
 - Gauss-Seidel
 - SOR
- Krylov subspace methods
 - Conjugate Gradient
 - Preconditioned CG
 - Other CG like methods
- Multigrid method
- Direct methods
 - Sparse GE

Sparse Matrix Storage Schemes

- Diagonal
- Coordinate
- Ellpack-Itpack
- Compressed Sparse Row (CSR)

Diagonal Storage

- Diagonal storage is the most efficient and commonly used way of storing a structured or banded matrix.
- In this scheme, an $n \times n$ sparse matrix A containing d diagonals with nonzero entries is stored using the following two arrays:
 - An $n \times d$ array AA , that stores the nonzero diagonals.
 - A $d \times 1$ array $OFFSET$, that stores the offset of each diagonal with respect to the principal diagonal.

Standard Diagonal Storage

1	2	0	0	0	0
3	4	5	0	0	0
0	6	7	0	0	0
8	0	9	10	11	0
0	12	0	0	13	14
0	0	15	0	16	17

Sparse Matrix A

AA

—	—	1	2
—	3	4	5
—	6	7	0
8	9	10	11
12	0	13	14
15	16	17	—

OFFSET

-3	-1	0	1
----	----	---	---

Storage in
Diagonal Format

Custom matlab function

`A = my_spdiags(AA,OFFSET,n)`

Row numbers in A correspond to row numbers in AA

Column-Oriented Diagonal Storage (spdiags)

1	2	0	0	0	0
3	4	5	0	0	0
0	6	7	0	0	0
8	0	9	10	11	0
0	12	0	0	13	14
0	0	15	0	16	17

Sparse Matrix A

AA

8	3	1	—
12	6	4	2
15	9	7	5
—	0	10	0
—	16	13	11
—	—	17	14

Storage in Column
Diagonal Format

OFFSET

—3	—1	0	1
----	----	---	---

Matlab function

`A = spdiags(AA,OFFSET,n,n)`

Column numbers in A correspond to row numbers in AA

Coordinate Storage

- Coordinate storage is the simplest and most straightforward way of storing an unstructured matrix.
- In this scheme, an $n \times n$ sparse matrix A containing q nonzero entries is stored using the following three arrays:
 - A $q \times 1$ array AA , that contains the nonzero elements. These entries can be stored in any order.
 - A $q \times 1$ array IA , that stores the row numbers of each nonzero element.
 - A $q \times 1$ array JA , that stores the column numbers of each nonzero element.

Coordinate Storage

$$\begin{bmatrix} 2. & 3. & 0 & 0 & 0 & 0 \\ 1. & 6. & 5. & 0 & 0 & 0 \\ 0 & 2. & 7. & 0 & 0 & 0 \\ 10. & 0 & 9. & 1. & 5. & 0 \\ 0 & .5 & 0 & 0 & 1.3 & 1. \\ 0 & 0 & 1.2 & 0 & 3. & 4. \end{bmatrix}$$

Sparse Matrix A

AA

2.	3.	1.	6.	5.	2.	7.	10.	9.	1.	5.	.5	1.3	1.	1.2	3.	4.
----	----	----	----	----	----	----	-----	----	----	----	----	-----	----	-----	----	----

IA

1	1	2	2	2	3	3	4	4	4	4	5	5	5	6	6	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

JA

1	2	1	2	3	2	3	1	3	4	5	2	5	6	3	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Storage in Coordinate format

Matlab function
`A = sparse(IA,JA,AA)`

Ellpack-Itpack Storage

- Ellpack-Itpack storage is suitable for general sparse matrices in which the maximum number of nonzero elements in any row is not much larger than the average number of nonzero elements per row.
- In this scheme, an $n \times n$ sparse matrix A for which the maximum number of nonzero elements in any row is m is stored using two $n \times m$ arrays:
 - An $n \times m$ array AA , each row of which contains the nonzero elements of the corresponding row of A .
 - An $n \times m$ array JA , that stores the column numbers of the corresponding entries in AA .

Ellpack-Itpack Storage

$$\begin{bmatrix} 1 & 2 & 0 & 0 & 0 & 0 \\ 3 & 4 & 5 & 0 & 0 & 0 \\ 0 & 6 & 7 & 0 & 0 & 0 \\ 8 & 0 & 9 & 10 & 11 & 0 \\ 0 & 12 & 0 & 0 & 13 & 14 \\ 0 & 0 & 15 & 0 & 16 & 17 \end{bmatrix}$$

Sparse Matrix A

AA

1	2	0	0
3	4	5	0
6	7	0	0
8	9	10	11
12	13	14	0
15	16	17	0

JA

1	2	-1	-1
1	2	3	-1
2	3	-1	-1
1	3	4	5
2	5	6	-1
3	5	6	-1

Storage in Ellpack-Itpack Format

Compressed Sparse Row (CSR) Storage

- CSR is the most efficient and commonly used way of storing an unstructured matrix.
- Let A be a $n \times n$ sparse matrix containing q nonzero entries. The following three arrays store the matrix in CSR format:
 - A $q \times 1$ array AA , that contains the nonzero elements. These are stored in the order of their rows from 1 to n ; however, elements of the same row can be stored in any order.
 - A $q \times 1$ array JA , that stores the column numbers of each nonzero element.
 - An $n \times 1$ array IA , the i^{th} entry of which points to the first entry of the i^{th} row in AA and JA .

Compressed Sparse Row (CSR)

Storage

$$\begin{bmatrix} 2. & 3. & 0 & 0 & 0 & 0 \\ 1. & 6. & 5. & 0 & 0 & 0 \\ 0 & 2. & 7. & 0 & 0 & 0 \\ 10. & 0 & 9. & 1. & 5. & 0 \\ 0 & .5 & 0 & 0 & 1.3 & 1. \\ 0 & 0 & 1.2 & 0 & 3. & 4. \end{bmatrix}$$

Sparse Matrix A

AA

2.	3.	1.	6.	5.	2.	7.	10.	9.	1.	5.	.5	1.3	1.	1.2	3.	4.
----	----	----	----	----	----	----	-----	----	----	----	----	-----	----	-----	----	----

JA

1	2	1	2	3	2	3	1	3	4	5	2	5	6	3	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

IA

1	3	6	8	12	15
---	---	---	---	----	----

Storage in CSR

Jacobi Iterative Scheme

- Jacobi iterative scheme is a simple iterative method to solve $Ax=b$ for x .
- It can be applied if all diagonal elements of A are non-zero.
- The k^{th} iteration of Jacobi can be expressed as

$$x_k(i) = \frac{1}{A(i,i)} \left(b(i) - \sum_{j=1}^n A(i,j)x_{k-1}(j) \right) + x_{k-1}(i)$$

Jacobi Scheme (contd.)

- Jacobi Scheme can be derived as follows:

$$Ax = b$$

$$b - Ax = 0$$

$$b - [A - D + D]x = 0$$

$$b - [A - D]x_{k-1} - Dx_k \approx 0 \quad (k \text{ is the iteration number})$$

$$x_k = D^{-1}b - D^{-1}[A - D]x_{k-1}$$

$$x_k(i) = \frac{1}{A(i,i)} \left(b(i) - \sum_{\substack{j=1 \\ j \neq i}}^n A(i,j)x_{k-1}(j) \right)$$

$$x_k(i) = \frac{1}{A(i,i)} \left(b(i) - \sum_{j=1}^n A(i,j)x_{k-1}(j) \right) + x_{k-1}(i)$$

Jacobi Iteration Algorithm

- A single row equation for the matrix equation $\mathbf{Ax}=\mathbf{b}$ can be rewritten as

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j \right)$$

- By adding and subtracting the term $a_{ii}x_i$ to the summation term above we arrive at the k^{th} Jacobi iteration given by

$$x_i^k = x_i^{k-1} + \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^n a_{ij} x_j^{k-1} \right) \quad \text{for } i = 1, \dots, n \text{ and } k = 1, \dots, \text{converged}$$

Jacobi Method (contd.)

- Jacobi iteration is a very simple method for solving a system of linear or non-linear equations.
- Advantages
 - Easy to implement
 - Can be used for linear and non-linear systems
 - Low storage requirements (for sparse matrices from FDA)
- Disadvantages
 - Convergence only guaranteed for diagonally dominant matrices (linear case)

$$|a_{ii}| = \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad \text{for all } i$$

- Very slow convergence

Improvements to Jacobi

- Gauss-Siedel
- Successive Over Relaxation (SOR)

Gauss-Seidel Scheme

- A slight improvement over Jacobi
 - Uses “most recent values” of x that are available, since these are probably more accurate
- The k^{th} iteration of Gauss-Seidel can be expressed as

$$x_k(i) = x_{k-1}(i) + \frac{1}{A(i,i)} \left(b(i) - \sum_{j=1}^{i-1} A(i,j)x_k(j) - \sum_{j=i}^n A(i,j)x_{k-1}(j) \right)$$

- G-S generally converges twice as fast as Jacobi

SOR

- Successive Over Relaxation (SOR) is a slight improvement over G-S:
 - A weighted average of $x_k(i)$ and $x_{k-1}(i)$ is used.
 - The weighting factor ω is typically $0 < \omega < 2$.

$$x_k(i) = \frac{\omega}{A(i,i)} \left(b(i) - \sum_{j=1}^{i-1} A(i,j)x_k(j) - \sum_{j=i}^n A(i,j)x_{k-1}(j) \right) + (2 - \omega)x_{k-1}(i)$$

- ω can be varied as iterations proceed.
- Block schemes can also be derived for Jacobi, G-S, and SOR.

Conjugate Gradient Method

- CG is a minimization based method for solving $Ax=b$ where A is *symmetric* and *positive definite* ($x^T Ax > 0$ for any real non-zero x).
- For an SPD matrix A , the unique x that minimizes the quadratic function $q(x) = 1/2 x^T Ax - x^T b$ is the solution to the system $Ax=b$.
- Each iteration of the CG algorithm consists of computing a scalar step size α_k and a direction vector p_k .
- The update operation is $x_k = x_{k-1} + \alpha_k p_k$

CG Algorithm

Procedure CG

$k=0; x_0=0; r_0=b; \alpha_0 = ||r_0||^2$

Do while ($\alpha_k > \epsilon ||r_0||$)

if ($k = 0$) then

$p_1 = r_0$

else

$p_{k+1} = r_k + \alpha_k p_k / \alpha_{k-1}$ // saxpy

endif

$k = k+1$

$w_k = A p_k$ // spmatvec

$\alpha_k = \alpha_{k-1} / p_k^T w_k$ // dot product

$x_k = x_{k-1} + \alpha_k p_k$ // saxpy

$r_k = r_{k-1} - \alpha_k w_k$ // axpy

$\alpha_k = ||r_k||^2$ // 2-norm (dot product)

Endwhile

$x = x_k$

End CG

Preconditioned Conjugate Gradient

- CG algorithm may require m iterations to converge if the matrix A has m distinct Eigen values.
- Convergence may be slow if A has a large number widely varying distinct Eigen values.
- Preconditioning is applied to matrix A to improve convergence.
 - We solve $M^{-1}[Ax]=M^{-1}[b]$ instead of $Ax=b$.
- The preconditioner M is an approximation of A but the operation $M^{-1}r$ should not be costly.
- The resulting algorithm is called PCG.

PCG Algorithm

Procedure CG

$k=0; x_0=0; r_0=b; \alpha_0 = ||r_0||^2$

Do while ($\alpha_k > \epsilon ||r_0||$)

solve the system $Mz_k = r_k$ //preconditioning

$\beta_k = r_k^T z_k$ //dot product

if ($k = 0$) then

$p_1 = z_0$

else

$p_{k+1} = z_k + \beta_k p_k / \beta_{k-1}$ //saxpy

endif

$k = k+1$

$w_k = Ap_k$ //spmatvec

$\alpha_k = \beta_{k-1} / p_k^T w_k$ //dot product

$x_k = x_{k-1} + \alpha_k p_k$ //saxpy

$r_k = r_{k-1} - \alpha_k w_k$ // saxpy

$\alpha_k = ||r_k||^2$ //2-norm (dot product)

Endwhile

$x = x_k$

End CG

Basic Operations in PCG

- 3 Saxpy's (simple ax plus y)
 - $6n$ ops
- 3 vector inner products (dot products)
 - $6n$ ops
- 1 spmatvec
 - $2 * (\text{average non-zeros per row}) * n$ ops
- Solve the system $Mz_k = r_k$
 - The complexity can vary significantly based on what we choose as M .
 - For example if we choose $M = \text{diag}(A)$ then the number of operations is n .

Other CG like algorithms

- CG algorithm is only applicable to SPD matrices.
- Other CG like iterative algorithms are available for sparse non-symmetric systems
 - BiCGSTAB
 - GMRES
 - QMR
 - CGS
 - ORTHOMIN
- All of the above algorithms require similar operations as CG
 - Dot products, spmatvecs, saxpy's, preconditionings etc.

Multigrid Method

- Multigrid gets its efficiency by solving the problem on a coarser grid and then using this solution to improve convergence of the fine grid solution.
- Basic Algorithm:
 - Replace problem on fine grid by an approximation on a coarser grid
 - Solve the coarse grid problem approximately, and use the solution as a starting guess for the fine-grid problem, which is then iteratively updated
 - Solve the coarse grid problem **recursively**, i.e. by using a still coarser grid approximation, etc.
- Success depends on coarse grid solution being a good approximation to the fine grid.

Multigrid Methods (contd.)

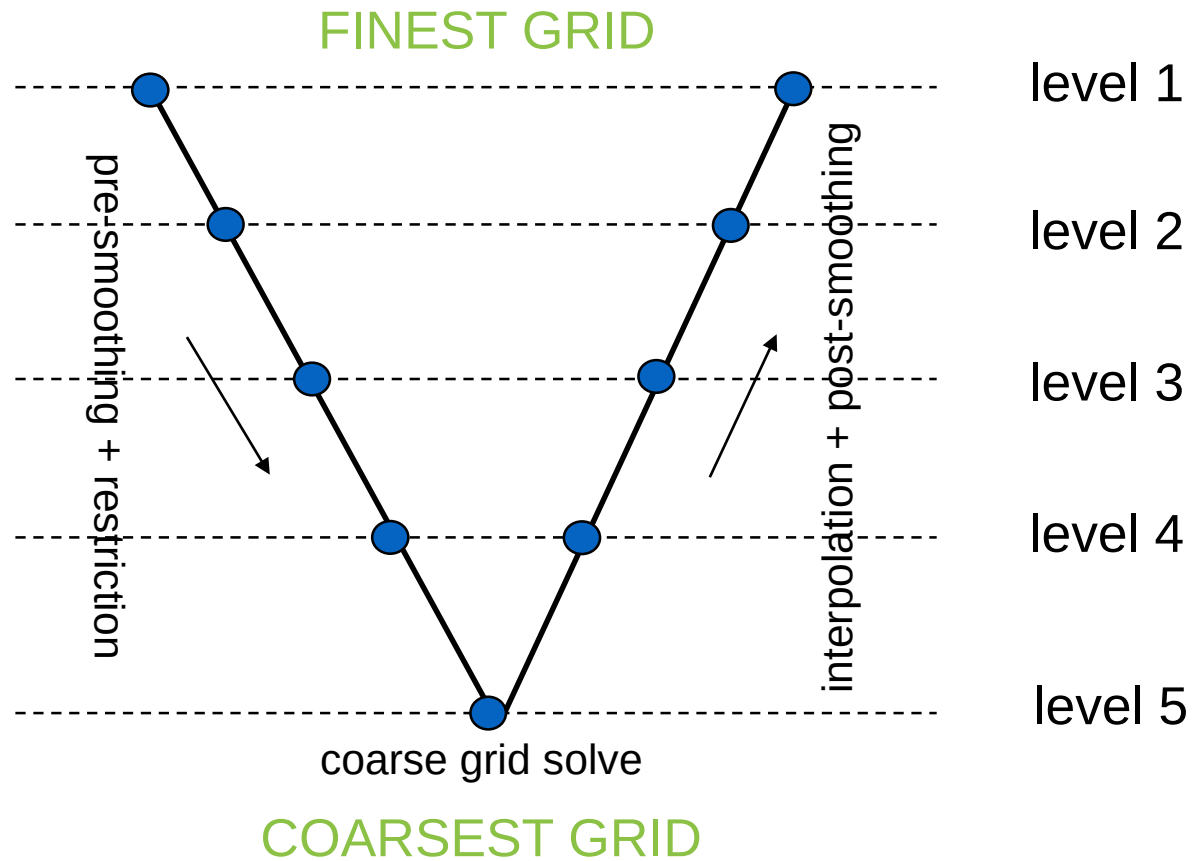
- Advantages

- More efficient by not attempting to resolve everything at the finest grid.
- Less sensitive to problem size.

- Disadvantages

- Common versions restricted to uniform grids.
- Difficult to implement on massively parallel machines.
- Cannot be easily used as an independent matrix solver (exception: algebraic multigrid).
- Requires more memory.

Multigrid V-Cycle



Solution Efficiency with Problem Size

NP	$n_x \times n_y \times n_z$	MG		DPCG	
		Iter	time	Iter	time
1	33 x 33 x 65	6	21.1	98	42.3
2	65 x 33 x 65	6	21.2	147	61.0
4	65 x 65 x 65	6	21.2	151	63.6
8	129 x 65 x 65	6	21.8	245	98.7
16	129 x 129 x 65	6	22.2	242	98.4
32	257 x 129 x 65	6	23.3	358	142
64	257 x 257 x 65	6	23.7	429	171
128	513 x 257 x 65	6	26.5	664	260
256	513 x 513 x 65	6	28.5	756	299
512	1025 x 513 x 65	6	35.5	1203	469
1024	1025 x 1025 x 65	6	39.2	1612	670

Number of pre- and post- smoothings = 3

Number of multigrid levels = 4

Effect of Heterogeneity

σ	MG		MGCG		DPCG	
	Iter	Time	Iter	Time	Iter	Time
0.0	6	12.0	10	22.2	429	52.3
0.5	6	13.1	7	19.0	610	72.8
1.0	9	18.9	9	22.5	665	79.2
1.5	13	29.0	9	23.5	731	86.7
2.0	18	36.4	11	27.2	804	95.2
2.5	33	65.8	15	35.2	883	104.2
5.0	Diverged		62	127.2	1392	157.2

Problem Size = 257 x 257 x 65

Number of Processors = 256

Number of Multigrid Levels =4

Number of Pre- and Post- smoothings =3

Sparse Gaussian Elimination

- Matlab's backslash operator ' $\backslash$ ' uses sparse GE if the matrix is defined to be sparse.
- Sparse GE is very similar to dense GE except that we omit multiplication by zero entries and store the matrix in a sparse (usually band storage) form.
 - Reduce $Ax=b$ to $Ux=y$ and solve for x using back substitution.
- A symbolic factorization is usually required prior to factorization to determine the sparsity pattern of the decomposed matrix.

Sparse Gaussian Elimination

- In sparse GE, pivot rows need not be chosen sequentially since many rows can be mutually independent and the update operation can be performed simultaneously for these rows.
- Symbolic factorization:
 - When the k^{th} row of the coefficient matrix A is used as the pivot, a fill-in is created for every $A(i,j)$ if $i, j > k$, $A(i,k) \neq 0$ and $A(k,j) \neq 0$. i.e.,

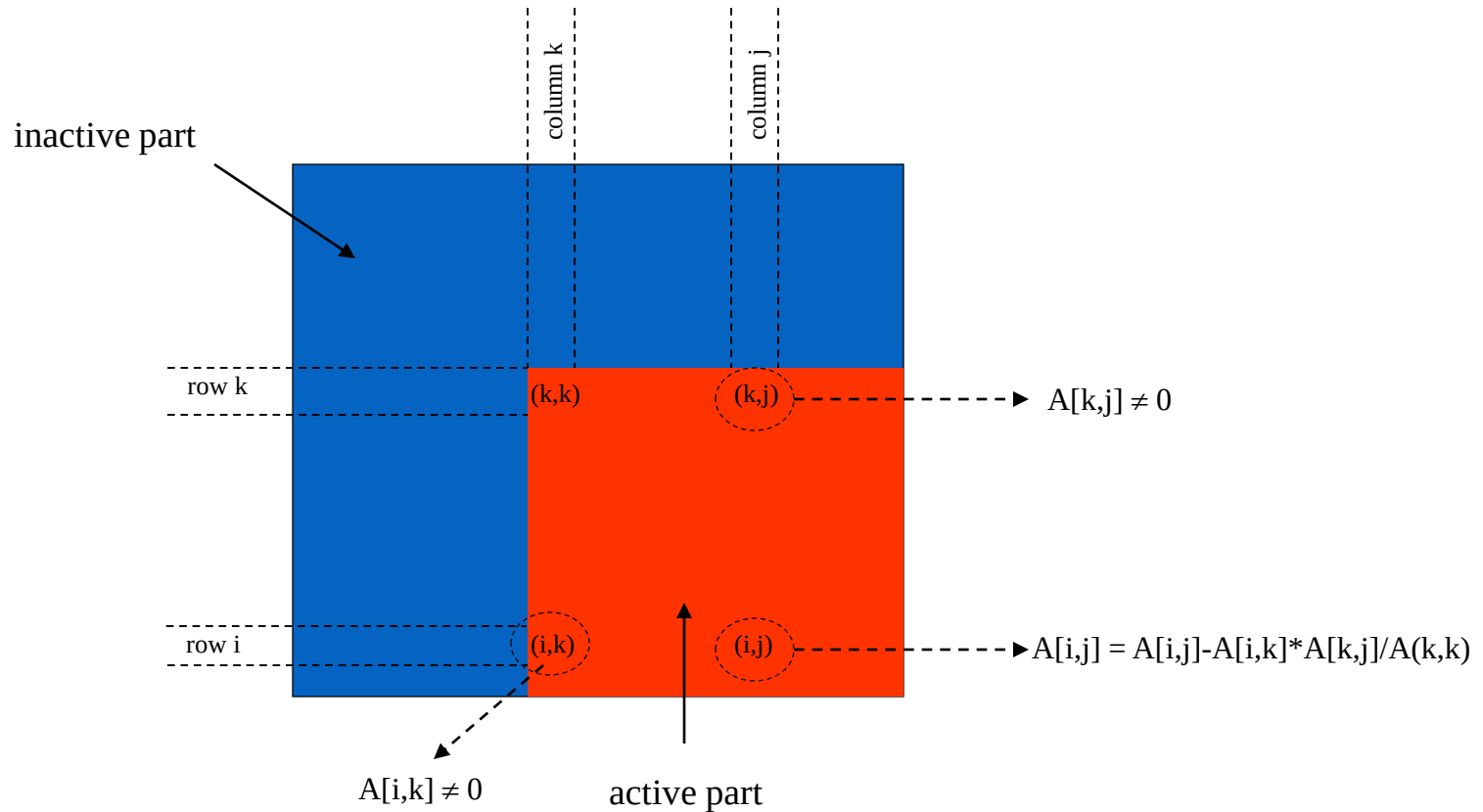
$$A(i, j) \neq 0 \text{ if } i, j > k, A(i, k) \neq 0 \text{ and } A(k, j) \neq 0.$$

Sparse Gaussian Elimination Overview

- Subtract multiples of each row to later rows to make **A** upper triangular
- Solve resulting upper triangular system **Ux = y** by backsubstitution

```
// for each column k
// zero it out below the diagonal by adding multiples of row k to later rows
for k = 1 to n
    // for each row i below row k
    for i = k+1 to n
        // subtract a multiple of row i from row k
        // for all columns j to the right of column k
        for j = k+1 to n
            if (A(k,j) & A(i,k) ≠ 0 )
                A(i,j) = A(i,j) - (A(k,j)/A(k,k)) * A(i,k)
            endifor
        endfor
    endfor
endfor
```

Sparse GE (contd.)



Sparse Gaussian Elimination

```
// Sparse GE Algorithm – factor A into LU
// Computes L and U and stores into A
Procedure SGE(A)
// S is a data structure to store sparsity
for i = 1 to n
    Si = 0
endfor
for k = 1 to n
    for j = 1 to k-1
        if (j is in Sk) then
            modify_gauss[k,A(j,*)]
        endif
    endfor
    divide_gauss[k,A(k,*)]
// the loop below is redundant since S is usually
// pre-computed and A is stored accordingly
    for i = k+1 to n
        if (A(i,k) ≠ 0) then
            include k in Si
        endif
    endfor
endfor
end Procedure SGE
```

```
// perform update operation on non-zero
// entries of row k
```

```
Procedure modify_gauss[i,A(k,*)]
for j = k+1 to n
    if A(k,j) ≠ 0 then
        A(i,j) = A(i,j) - A(i,k)*A(k,j)
    endif
endfor
end Procedure modify_gauss
```

```
// perform division operation on non-zero
// entries of row k
```

```
Procedure divide_gauss[k,A(k,*)]
for j = k+1 to n
    if A(k,j) ≠ 0 then
        A(k,j) = A(k,j)/A(k,k)
    endif
endfor
end Procedure divide_gauss
```

MATLAB functions for linear system solution

- Backslash operator ' $\backslash$ '
 - To solve $Ax=b$ for x , use $x=A\backslash b$
 - Uses Gaussian Elimination with pivoting
 - Sparse GE for sparse matrices
- Iterative methods (use help to get detailed info)
 - PCG: $x=pcg(A,b)$ } For symmetric SPD matrices
 - GMRES: $x=gmres(A,b)$
 - BICGSTAB: $x=bicgstab(A,b)$
 - CGS: $x=cgs(A,b)$
 - QMR: $x=qmr(A,b)$ } For general sparse matrices

MATLAB example for linear system solution

- Solve the matrix resulting from FDA of groundwater flow equation (2-D linear Poisson equation) using various methods (linear_solver_example.m, gwflow2d_matrix.m):

$$k_x \frac{\partial^2 h}{\partial x^2} + k_y \frac{\partial^2 h}{\partial y^2} = q(x, y)$$

Solving Nonlinear Systems

- Applications
- Methods

Applications

- Nonlinear systems of equations commonly arise in many engineering applications
- Solving nonlinear PDE systems by finite-differences or finite-elements
- Engineering optimization with constraints

Common Methods

- Newton-Raphson method
- Reduced Newton methods
- Linearization (e.g., Picard iteration).
- Minimization based methods

Solving nonlinear system of equations by Newton-Raphson method

- Solve $\mathbf{F}(\mathbf{x})=0$, where $\mathbf{x}$ is a vector with n variables: $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$, and F is a vector function of n functions.
- i.e.,

$$f_1(x_1, x_2, \dots, x_n) = 0,$$

$$f_2(x_1, x_2, \dots, x_n) = 0,$$

$$\vdots$$

$$f_n(x_1, x_2, \dots, x_n) = 0$$

Newton-Raphson Method - Scalar

- Newton-Raphson method expands the nonlinear system of equations as a multi-dimensional Taylor series and neglects all second order terms

$$f(x_k + \delta x_k) = f(x_k) + f'(x_k)\delta x_k + \dots$$

setting $x_k + \delta x_k = x_{k+1}$ and neglecting 2nd order terms we get

$$f(x_{k+1}) = f(x_k) + f'(x_k)(x_{k+1} - x_k)$$

setting $f(x_{k+1}) = 0$ we get, $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$, or

$$\begin{aligned}\delta x_k &= -[f'(x_k)]^{-1} f(x_k) \\ x_{k+1} &= x_k + \delta x_k\end{aligned}$$

Newton-Raphson Method - Vector

- Newton-Raphson method expands the nonlinear system of equations as a multi-dimensional Taylor series and neglects all second order terms

$$\mathbf{F}(\mathbf{x}^k + \Delta\mathbf{x}^k) = \mathbf{F}(\mathbf{x}^k) + \mathbf{F}'(\mathbf{x}^k)\Delta\mathbf{x}^k + \dots$$

setting $\mathbf{x}^k + \Delta\mathbf{x}^k = \mathbf{x}^{k+1}$ and neglecting 2nd order terms we get

$$\mathbf{F}(\mathbf{x}^{k+1}) = \mathbf{F}(\mathbf{x}^k) + \mathbf{F}'(\mathbf{x}^k)\Delta\mathbf{x}^k$$

setting $\mathbf{F}(\mathbf{x}^{k+1}) = 0$ we get,

$$\Delta\mathbf{x}^k = -[\mathbf{F}'(\mathbf{x}^k)]^{-1} \mathbf{F}(\mathbf{x}^k)$$
$$\mathbf{x}^{k+1} = \mathbf{x}^k + \Delta\mathbf{x}^k$$

Jacobian Matrix $\mathbf{J}$

Newton-Raphson Method

- We start with an initial guess $\mathbf{x}^0$ and a linear system involving $\mathbf{J}$ is solved at the k^{th} iteration and the new value of $\mathbf{x}^{k+1}$ is computed as follows:

$$\text{Jacobian } \mathbf{J}(\mathbf{x}^k) \Delta^k = -\mathbf{F}(\mathbf{x}^k) \text{ rhs}$$

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \Delta^k$$

Newton-Raphson Method

- Newton-Raphson method requires the construction of the Jacobian matrix $\mathbf{J}$ and solve a linear system of equations involving $\mathbf{J}$.

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_j} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \dots & \vdots & \dots & \vdots \\ \frac{\partial f_i}{\partial x_1} & \dots & \frac{\partial f_i}{\partial x_j} & \dots & \frac{\partial f_i}{\partial x_n} \\ \vdots & \dots & \vdots & \dots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \dots & \frac{\partial f_n}{\partial x_j} & \dots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

- Each entry in matrix $\mathbf{J}$ can be calculated by finite-differences or analytically. This requires $2n^2$ evaluations of f if central differencing is used.

Computation of Jacobian Matrix

- If possible, the Jacobian matrix should be computed analytically.
- Finite differencing can be used if the function $\mathbf{F}$ is not an analytical function (e.g., discrete approximation to a complicated PDE)

$$\frac{\partial f_i^k(x_1^k, x_2^k, \dots, x_j^k, \dots, x_n^k)}{\partial x_j} = \frac{f_i^k(x_1^k, x_2^k, \dots, x_j^k + \delta x^k, \dots, x_n^k) - f_i^k(x_1^k, x_2^k, \dots, x_j^k - \delta x^k, \dots, x_n^k)}{2\delta x^k}$$

Reduced Newton Methods

- Computation and storage of Jacobian can be expensive for dense problems
- Reduced Newton methods compute only the diagonal or a few off-diagonals of the Jacobian matrix.
- If the Jacobian is diagonally dominant then this can result in tremendous computational and memory savings.

MATLAB function for nonlinear system solution

- `fsolve` function
 - `fsolve(fun, x0)`
 - `fsolve` handles both square and rectangular systems (least squares problem)
 - Uses optimization algorithms
 - Gauss-Newton, Levenberg-Marquadt, conjugate gradient trust region (`cgtrust`)
 - Default is `cgtrust`
 - User may supply the Jacobian

Nonlinear System solution Example

- Solve the following system of equations for x_1, x_2, x_3

$$e^{2x_1} - x_2 - 4 = 0$$

$$x_2 - x_3^2 - 1 = 0$$

$$x_3 - \sin x_1 = 0$$

- Use Newton-Raphson method and `fsolve` to solve this problem

Newton-Raphson method for example problem

- Jacobian Matrix is given by

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} \end{bmatrix} = \begin{bmatrix} 2e^{2x_1} & -1 & 0 \\ 0 & 1 & -2x_3 \\ -\cos x_1 & 0 & 1 \end{bmatrix}$$

- Solution is given by the following steps

$$\mathbf{J}(\mathbf{x}^k) \Delta^k = -\mathbf{F}(\mathbf{x}^k)$$

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \Delta^k$$

Newton-Raphson method for example problem

- Solution is given by the following iteration steps

$$\begin{bmatrix} dx_1 \\ dx_2 \\ dx_3 \end{bmatrix}^{(k)} = -\mathbf{J}^{-1} \begin{bmatrix} f_1(x_1, x_2, x_3) \\ f_2(x_1, x_2, x_3) \\ f_3(x_1, x_2, x_3) \end{bmatrix}^{(k)}$$

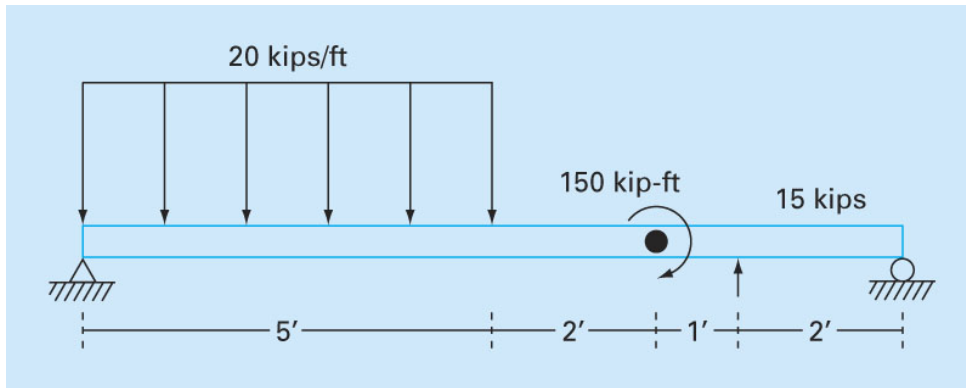
$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}^{(k+1)} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}^{(k)} + \begin{bmatrix} dx_1 \\ dx_2 \\ dx_3 \end{bmatrix}^{(k)}$$

MATLAB code for example

- MATLAB code for this example is in:
 - [nonlinear system example.m](#)

Example1 - C&C 8.24, p. 219

beam_example.m



Find the points x where shear $V = 0$

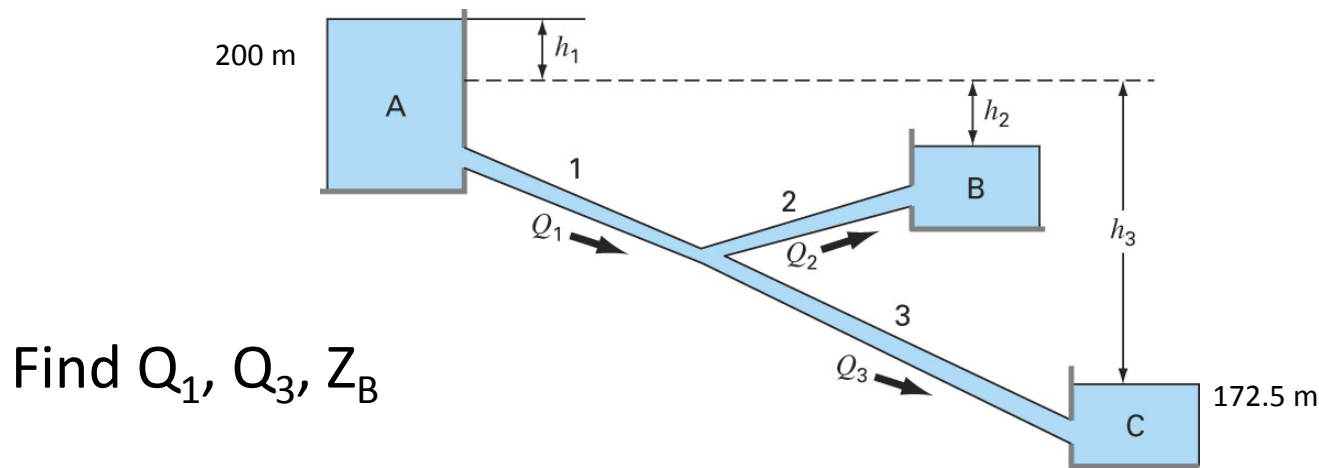
$$V(x) = 20 \left[\langle x - 0 \rangle^1 + \langle x - 5 \rangle^1 \right] - 15 \langle x - 8 \rangle^0 - 57$$

singularity function:

$$\langle x - a \rangle^n = \begin{cases} (x - a)^n & \text{when } x > a \\ 0 & \text{when } x \leq a \end{cases}$$

Example 2 – C & C 8.44

pipe_example.m



Pipe	1	2	3
Length	1800	500	1400
Diameter	0.4	0.25	0.2
Flow	?	0.1	?

Contd..

$$\textcircled{2} - \frac{p_1}{\gamma} + \frac{V_1^2}{2g} + Z_1 - (h_{L1} + h_{L2}) = \frac{p_2}{\gamma} + \frac{V_2^2}{2g} + Z_2$$

$$\textcircled{3} - \frac{p_1}{\gamma} + \frac{V_1^2}{2g} + Z_1 - (h_{L1} + h_{L3}) = \frac{p_3}{\gamma} + \frac{V_3^2}{2g} + Z_3$$

$$h_{L2} = f \frac{L_2}{D_2} \cdot \frac{V_2^2}{2g} \quad / \quad V_2 = \frac{Q_2}{A_2}$$

Contd.

$$\frac{1}{f_2} = -2 \log \left(\frac{\Sigma}{3.7 D_2} + \frac{2.51}{Re_i \sqrt{f_i}} \right)$$

$$Re_i = \frac{V_i D_i}{\nu}$$

$$\Sigma = 0.0012 \text{ m}$$

$$\nu = 1 \times 10^{-6} \text{ m}^2/\text{s}$$